

CSE 101 Project Report

Students Grade System

Course: CSE 101

Semester: Spring 2026

Project Title: Students Grade System

Submitted to:

Student Name	Student Id
Md. Monirul Sarker Shadhin	2212145042
Monowar Hossain Aman	2321123042

1. Introduction

The project is an implementation of a Python based Students Grade System to keep student grade records. The program reads the student data in a text file, and puts the data in memory, then enables the instructor to do various grade related tasks using a menu and only when the user chooses to save and exit writes the updated data to the same file. This is in accordance with the project specifications that all processing should be carried out in memory, and only be saved at the end.

The input file contains student data in the form of studentID number student name number grades with the number symbol between the fields and the space between the grades. The program needs to be dynamic to any number of grades since there is no predetermined number of tests.

2. Objective of the Project

The overall goal of this project is to develop a menu driven system which can enable an instructor to effectively handle student grade information. The program should be able to display student records, search a given student, compute averages, update grades, add new test results, add new students, delete students and all changes properly. Another goal of the project is to apply the modular programming, input validation, file handling and adequate design of functions using the concepts discussed in class.

3. Problem Analysis

The specified issue will be addressed with a program that will read the student grade data on a file and offer eight menu options. All the options should be adopted as individual functions and the program needs to be left to run until a user chooses option 8. The program should also show appropriate error messages to invalid inputs like incorrect menu choice, student ID is not a number, quiz number is not a number, and the grade should be in the range of 0 to 100.

To address this issue, a list of dictionaries was taken. Every dictionary is a representation of a single student and has three keys: id, name and grades. This design was chosen since it is easy, comprehensible and adaptable. Grades field is a list, and it is easy to support as many tests as you want and do such operations as display, modification and average calculating. The source code that I uploaded adheres to this style.

4. Program Design

The program begins with reading all the student records in the text file in a list. Once the data has been loaded, the main menu will be shown, and the user will wait to choose an option. Both options invoke different functions which execute a particular task. When the job is finished, the control goes back to the menu. This process goes on until the user opts to save and quit and the altered data is copied to the same file. This design maintains the program in order and meets the need of modular programming without the use of global variables.

A sample student record inside the program looks like this:

```
{  
    "id": "2321123",  
    "name": "Aman",  
    "grades": [45.0, 20.0]  
}
```

This organization simplifies the management of the code since student information is easily accessed, and grade calculations could be made directly on the list of grades. This arrangement of information is reflected in the sample uploaded file.

5. Description of Functions

5.1 Load_from_file(filename)

This function will read the entire student records in the file at the start of the program. It divides each line by the symbol #, eliminates unnecessary spaces, changes grades into floating-point numbers, and stores each student in a dictionary as an element of a list. This is a significant role since it prepares the primary data structure on which the remainder of the program will operate.

5.2 Display_all_students(students)

This function realizes option 1. It shows the ID, name and all grades of all students. The test headings are created dynamically based on the number of grades available and this enables the program to be used with any number

of tests. Once the information has been displayed, the function waits until the Enter key is pressed, and then the menu is displayed again.

5.3 Display_one_student(students)

This is a function that executes option 2. It requires the user to input a student ID, searches and retrieves the student details and shows the student grade details to the user. In case that the student ID does not exist, then it displays a message of an error. In either instance, it awaits the Enter key and then goes back to the main menu.

5.4 Display_all_averages(students)

This function gives option 3. It averages the grades of the individual student by adding up the grades then dividing them by the number of grades. In case, the student has no grades, the program instead prints No tests. This guarantees the program that is functional even in case of missing grades.

5.5 Modify_student_grade(students)

This function implements option 4. The user inserts a student ID, quiz number and new grade into it. The role validates the student, quiz number and new grade is within the acceptable range. In case of all values being valid, the chosen grade is changed. The record prior to and subsequent modification is also displayed in the function.

5.6 Add_test_grades_for_all_students(students)

This option is option 5. It introduces a new test grade per student. The program will choose the next test number and then request the user to input a single grade on each student. All grades are checked to determine that they are numbers between 0 and 100. Bad inputs will generate an error message.

5.7 Add_new_student(students)

This function executes choice 6. It reads a new student ID, verifies the existence of the student ID and if not, it reads the name and grades of the student. This is followed by creating of a new dictionary and adding it to the list of students. In case the ID already exists, there is an error message displayed.

5.8 Delete_student(students)

This functionality carries out option 7. It finds the student ID the user has typed in. In case the student is located, the record is deleted off the list. Otherwise, the function gives an error message. This maintains the student list in correct run-time.

5.9 Save_to_file(students)

This functionality puts option 8 into effect. It puts back all amended records of students on the same file in the initial format. This action is invoked when the user selects Save and Exit, which is in accordance with the project instructions.

6. Main Menu Logic

A while True loop is used to control the menu. The program has the eight options inside the loop and reads the choice of the user. Each option is then called by an if-elif-else structure to call the appropriate function. In case the user enters an invalid option, a message of error would be shown. The program will cease upon the selection of option 8 which will save the data to the file. This is a plain design that is straightforward and a direct continuation of the project specification.

7. Input Validation and Error Handling

A significant portion of this project is input validation. The system verifies menu options are valid, the presence of a student ID, quiz number is in the right range, and the input grades are within the 0-100 range. Try and except are applied in certain functions to avoid crashing of the program when user enters the wrong type of data. These checks enhance the reliability of the system and meet the project requirement of valid type and valid range checking.

8. Modularity and Coding Style

The solution is based on modular programming by allocating each menu option to a given function. This simplifies the code to comprehend, debug and maintain. It was made more readable with meaningful variable names like student_id, quiz_number, new grade and found student. There were also comments made throughout the code to clarify the intention of significant steps, which is significant since comments are evaluated as part of the assessment criteria.

9. Screenshots of Running Program

The project brief asks for screenshots of the running program, so this section should include images taken from the actual execution of your code.

Insert the following screenshots:

Figure 1: Main menu of the Students Grade System

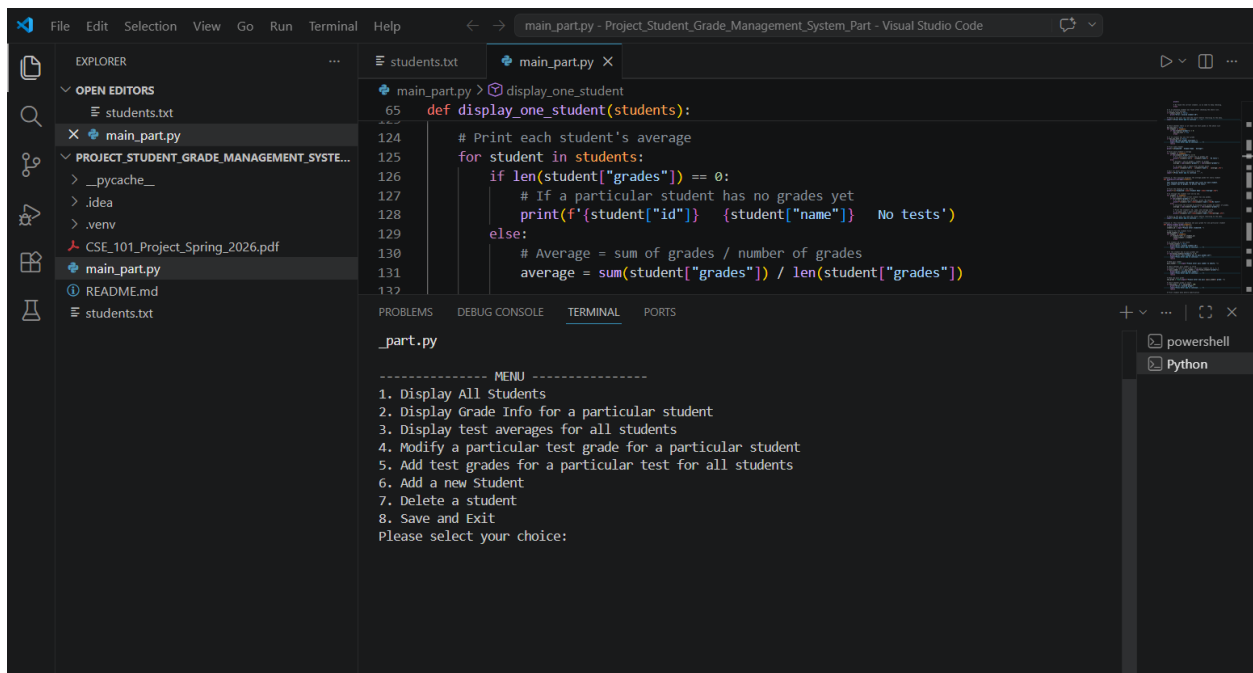
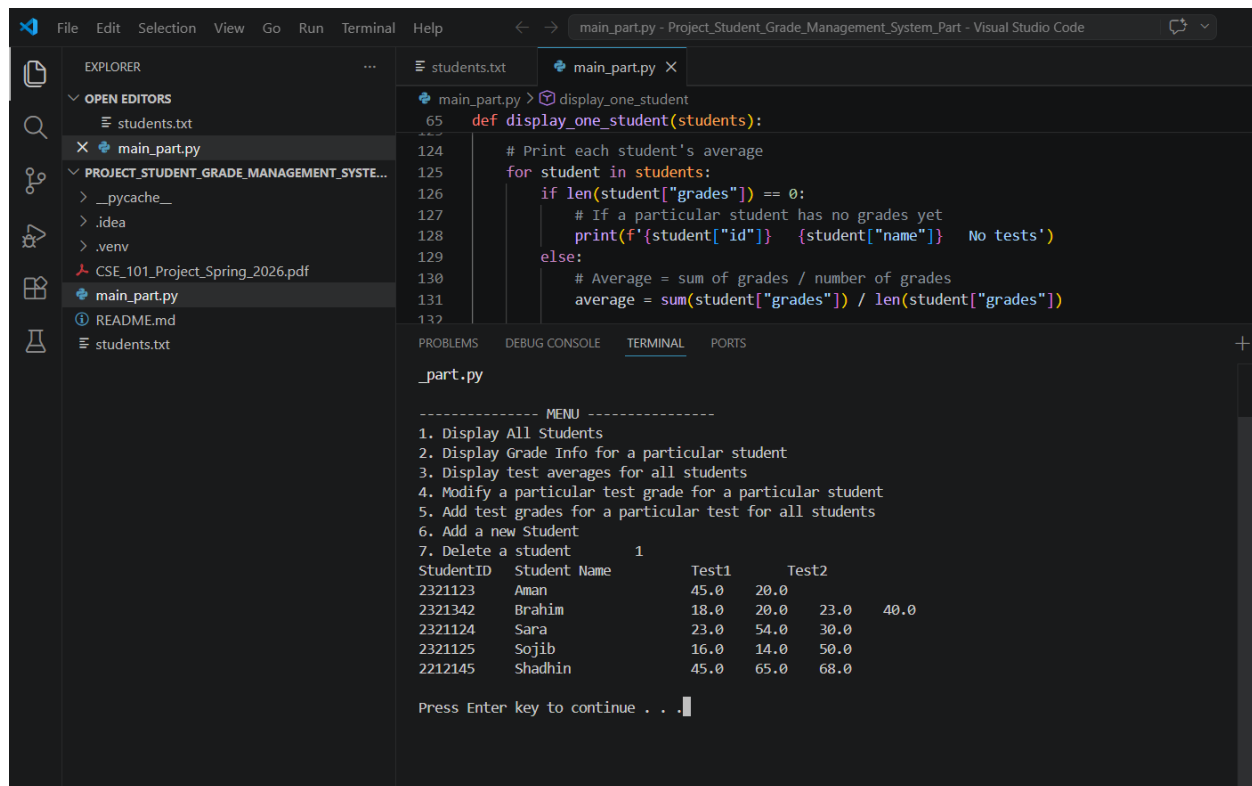


Figure 2: Option 1 showing all student records



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays the project structure, including files like `students.txt`, `main_part.py`, and `README.md`. The main editor window shows the `main_part.py` file with a function `display_one_student` that iterates through a list of students and prints their details. The output is visible in the terminal panel at the bottom.

```
def display_one_student(students):
    # Print each student's average
    for student in students:
        if len(student["grades"]) == 0:
            # If a particular student has no grades yet
            print(f'{student["id"]} {student["name"]} No tests')
        else:
            # Average = sum of grades / number of grades
            average = sum(student["grades"]) / len(student["grades"])
```

The terminal output shows a menu and a table of student records:

```
----- MENU -----
1. Display All Students
2. Display Grade Info for a particular student
3. Display test averages for all students
4. Modify a particular test grade for a particular student
5. Add test grades for a particular test for all students
6. Add a new Student
7. Delete a student      1

StudentID  Student Name  Test1  Test2
2321123   Aman         45.0   20.0
2321342   Brahim       18.0   20.0   23.0   40.0
2321124   Sara         23.0   54.0   30.0
2321125   Sojib        16.0   14.0   50.0
2212145   Shadhin      45.0   65.0   68.0

Press Enter key to continue . . .
```

Figure 3: Option 2 showing one student's information

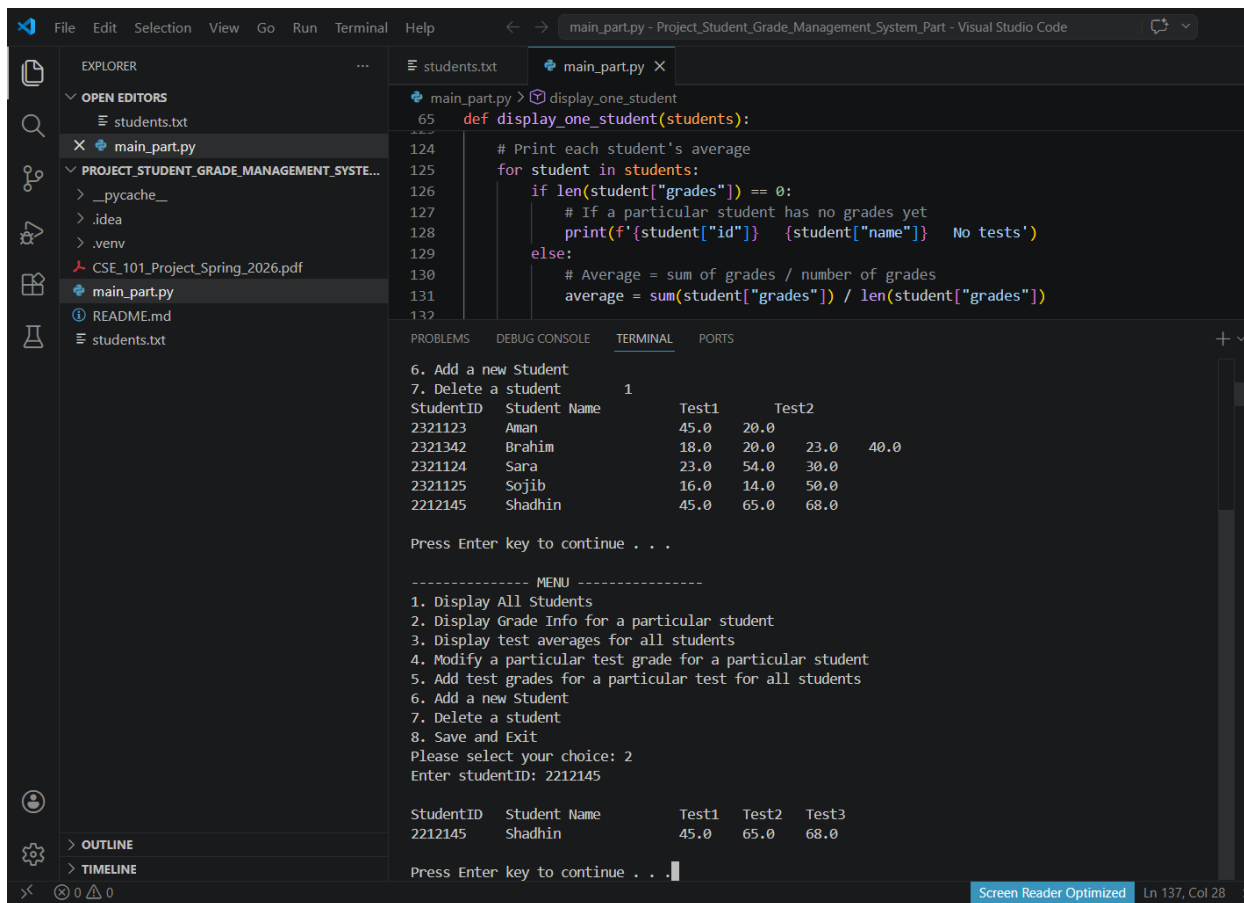


Figure 4: Option 3 showing average grades of all students


```
File Edit Selection View Go Run Terminal Help
main_part.py - Project_Student_Grade_Management_System_Part - Visual Studio Code

EXPLORER
OPEN EDITORS
students.txt
main_part.py
PROJECT_STUDENT_GRADE_MANAGEMENT_SYSTE...
_pycache_
.idea
.venv
CSE_101_Project_Spring_2026.pdf
main_part.py
README.md
students.txt

main_part.py > display_one_student
65 def display_one_student(students):
124 # Print each student's average
125 for student in students:
126     if len(student["grades"]) == 0:
127         # If a particular student has no grades yet
128         print(f'{student["id"]} {student["name"]} No tests')
129     else:
130         # Average = sum of grades / number of grades
131         average = sum(student["grades"]) / len(student["grades"])
132

PROBLEMS DEBUG CONSOLE TERMINAL PORTS
2321123 Aman 32.5
2321342 Brahim 25.2
2321124 Sara 35.7
2321125 Sojib 26.7
2212145 Shadhin 59.3

Press Enter key to continue . . . 3

----- MENU -----
1. Display All Students
2. Display Grade Info for a particular student
3. Display test averages for all students
4. Modify a particular test grade for a particular student
5. Add test grades for a particular test for all students
6. Add a new Student
7. Delete a student
8. Save and Exit
Please select your choice: 3

StudentID Student Name Average
2321123 Aman 32.5
2321342 Brahim 25.2
2321124 Sara 35.7
2321125 Sojib 26.7
2212145 Shadhin 59.3

Press Enter key to continue . . .
```

Figure 5: Option 4 before and after grade modification

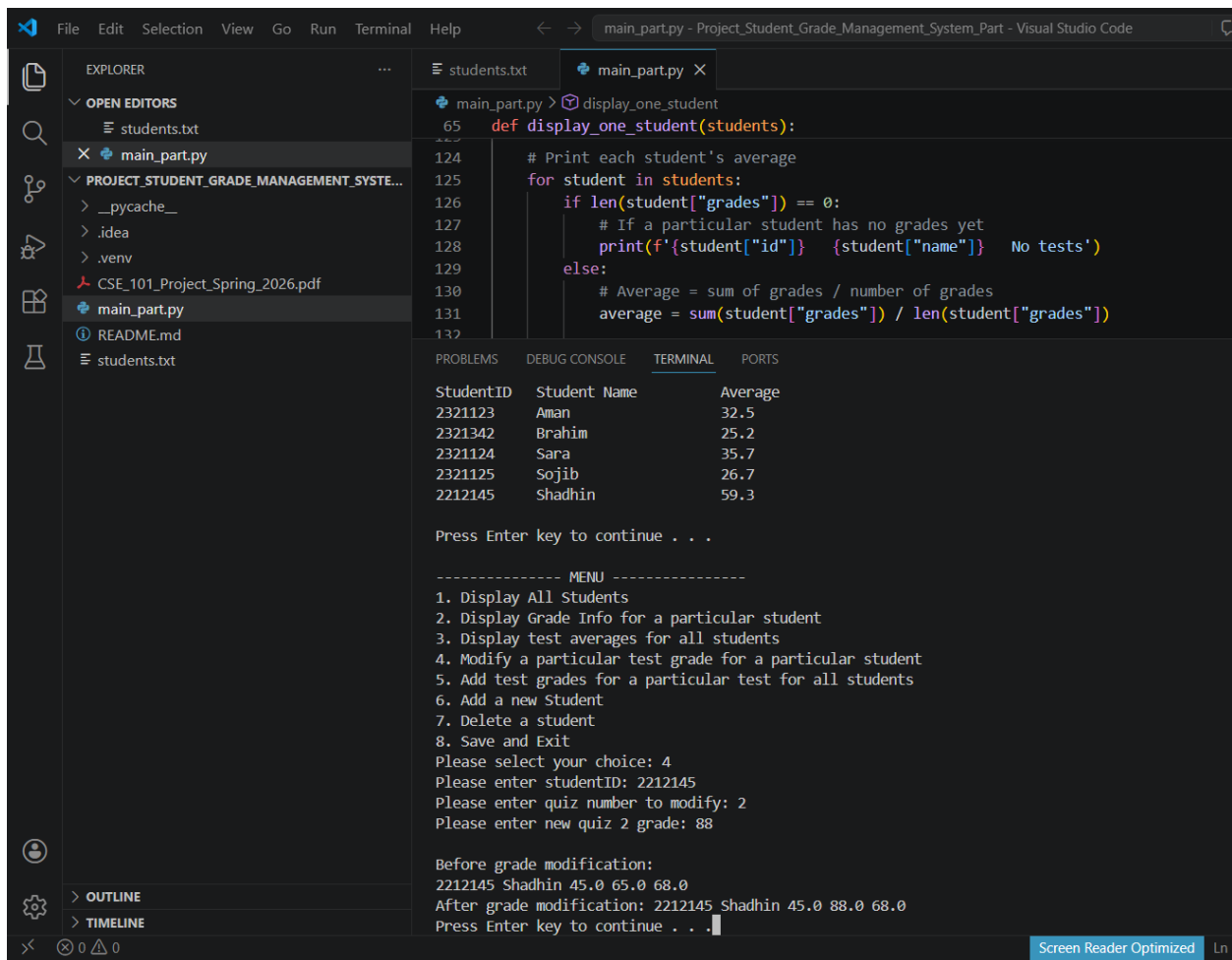


Figure 6: Option 5 adding new test grades

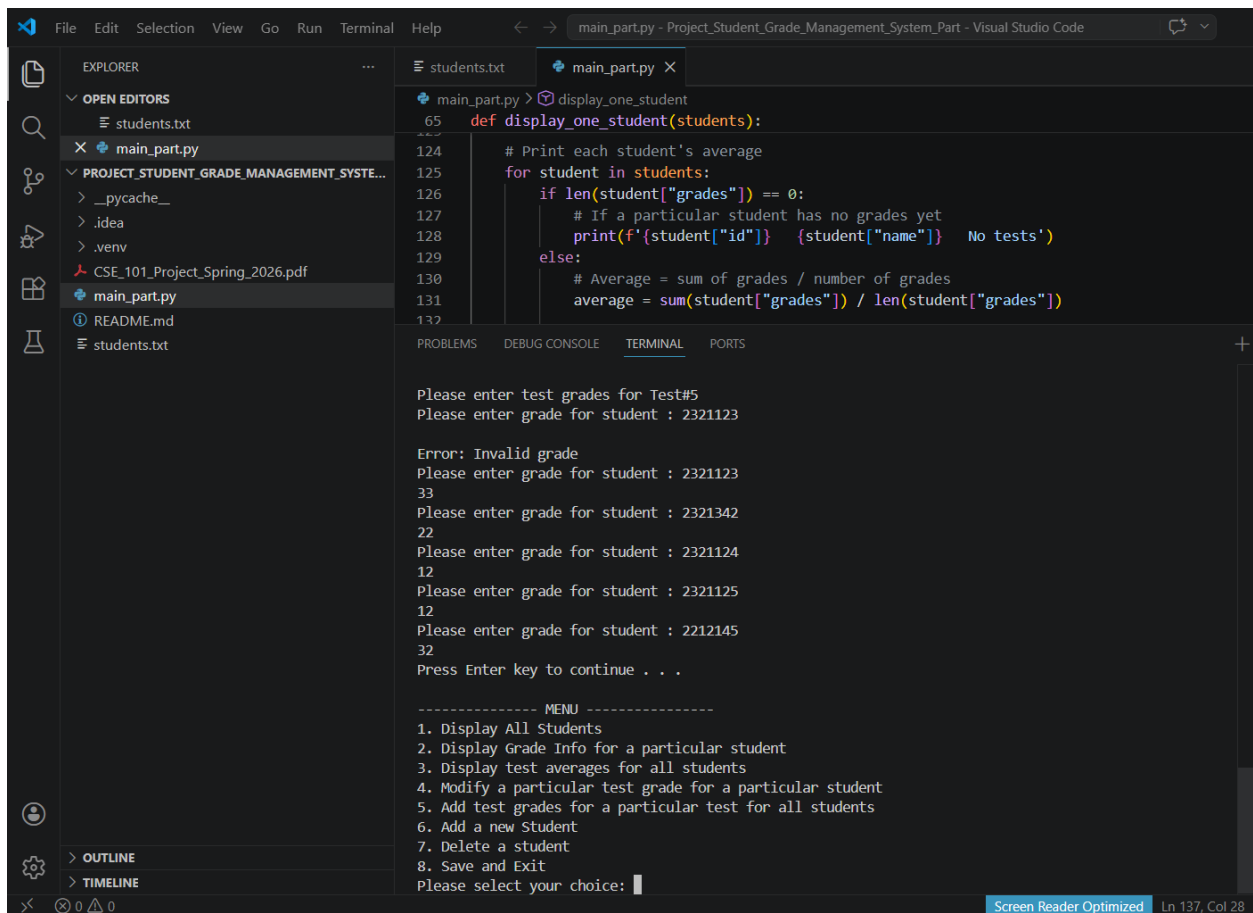


Figure 7: Option 6 adding a new student

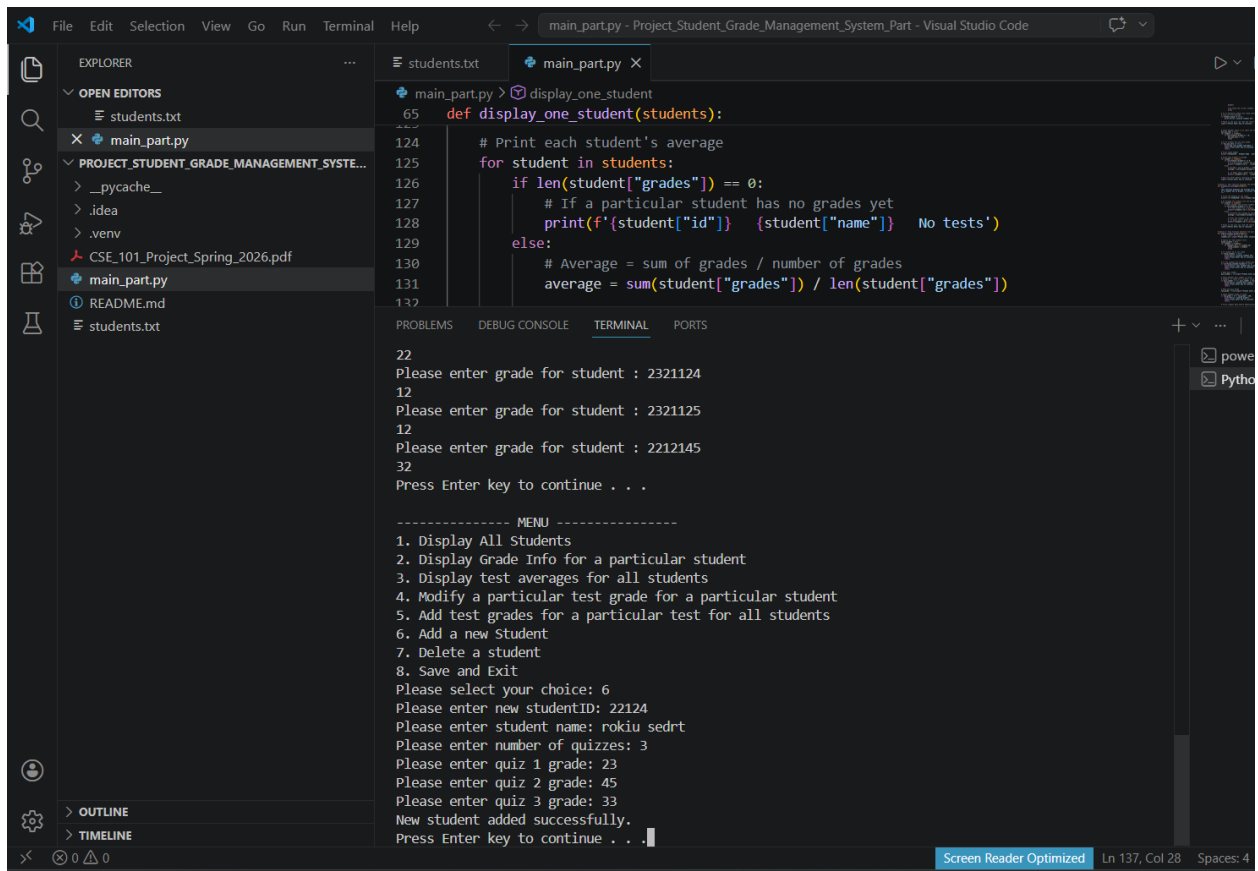


Figure 8: Option 7 deleting a student

The screenshot shows the Visual Studio Code interface with the file `main_part.py` open. The code defines a `display_one_student` function that iterates through a list of students and prints their details. The terminal window shows the program's execution, where the user has selected Option 8 (Save and Exit) from the menu. The program prompts for a student ID to delete, and the user enters 2212145. The program then confirms the deletion and prompts for the next action.

```

main_part.py > display_one_student
65 def display_one_student(students):
124     # Print each student's average
125     for student in students:
126         if len(student["grades"]) == 0:
127             # If a particular student has no grades yet
128             print(f'{student["id"]} {student["name"]} No tests')
129         else:
130             # Average = sum of grades / number of grades
131             average = sum(student["grades"]) / len(student["grades"])
132
PROBLEMS  DEBUG CONSOLE  TERMINAL  PORTS

5. Add test grades for a particular test for all students
6. Add a new Student
7. Delete a student
8. Save and Exit
Please select your choice: 6
Please enter new studentID: 22124
Please enter student name: rokiu sedrt
Please enter number of quizzes: 3
Please enter quiz 1 grade: 23
Please enter quiz 2 grade: 45
Please enter quiz 3 grade: 33
New student added successfully.
Press Enter key to continue . . .

----- MENU -----
1. Display All Students
2. Display Grade Info for a particular student
3. Display test averages for all students
4. Modify a particular test grade for a particular student
5. Add test grades for a particular test for all students
6. Add a new Student
7. Delete a student
8. Save and Exit
Please select your choice: 7
Please enter studentID to delete: 2212145
Student deleted successfully.
Press Enter key to continue . . .

```

Figure 9: Option 8 saving data and exiting

The screenshot shows the Visual Studio Code interface with the file `main_part.py` open. The code defines a `display_one_student` function. The terminal window shows the program's execution, where the user has selected Option 1 (Display All Students) from the menu. The program displays a table of student information, including their ID, name, and test scores. The user then selects Option 8 (Save and Exit) from the menu, and the program prompts for the next action.

```

main_part.py > display_one_student
65 def display_one_student(students):
124     # Print each student's average
125     for student in students:
126         if len(student["grades"]) == 0:
127             # If a particular student has no grades yet
128             print(f'{student["id"]} {student["name"]} No tests')
129         else:
130             # Average = sum of grades / number of grades
131             average = sum(student["grades"]) / len(student["grades"])
132
PROBLEMS  DEBUG CONSOLE  TERMINAL  PORTS

5. Add test grades for a particular test for all students
6. Add a new Student
7. Delete a student
8. Save and Exit
Please select your choice: 1
StudentID  Student Name      Test1    Test2    Test3
2321123    Aman              45.0     20.0     33.0
2321342    Brahim            18.0     20.0     23.0    40.0    22.0
2321124    Sara              23.0     54.0     30.0     12.0
2321125    Sojib             16.0     14.0     50.0     12.0
22124     rokiu sedrt       23.0     45.0     33.0

Press Enter key to continue . . .

----- MENU -----
1. Display All Students
2. Display Grade Info for a particular student
3. Display test averages for all students
4. Modify a particular test grade for a particular student
5. Add test grades for a particular test for all students
6. Add a new Student
7. Delete a student
8. Save and Exit
Please select your choice: 8
Data saved. Exiting program.
PS C:\Users\shadh\Downloads\Project_Student_Grade_Management_System_Part\Project_Student_Grade_Managem
ent_System_Part>

```

10. Challenges Faced

The variable number of grades was one of the challenges in this project as there is no specific number of tests defined in the file format. The second problem was the proper input validation to ensure that invalid IDs, grades and quiz numbers would not lead to wrongful results and program crashes. It was also important to save the updated information in the same file format as it might have a slight formatting error that would influence the execution of the program next time. Lists, dictionaries, loops, conditions and file handling were done with care and solved these challenges.

11. Conclusion

To sum up, this project is a success in the implementation of a Students Grade System in Python. The program will read the student data in a text file, store student data in a list of dictionaries, enables the user to manipulate records with eight menu options, check user input and save the modified records in the same file when the user exits. The project uses real world application of python basics like file manipulation, functions, loops, lists, dictionaries, and conditional statements. On the whole, the solution is able to satisfy the requirements of the given project and offers an efficient approach to the task of handling student grades.